

Reproduction and Detailed Experimental Analysis of CP-LSH: Content-Preserving Locality Sensitive Hashing for Graph Representation Learning

Bhavya Sri Kottana Sri Shraddha S Bhavya Sri Mamidi

Department of Computer Science
IIT Bombay

Abstract

Graph representation learning has become a central problem in modern machine learning due to the increasing availability of graph-structured data across domains such as citation networks, recommendation systems, and social media platforms. Traditional structure-based embedding methods often struggle with highly skewed degree distributions, where a large proportion of nodes have very few connections. In this project, we reproduce and analyze CP-LSH, a content-preserving graph embedding framework that integrates locality sensitive hashing with structural learning to address the long-tail problem. We conduct extensive experiments on citation datasets, compare against DeepWalk, analyze classification performance, tail-node behavior, link prediction accuracy, memory efficiency, runtime complexity, and sensitivity to hyperparameters. Our empirical findings confirm that CP-LSH improves representation quality for low-degree nodes while maintaining competitive overall performance.

Contents

1	Introduction	3
2	Background	3
3	Methodology	4
4	Experimental Setup	4
4.1	DBLP Dataset	4
5	Results and Analysis	5
5.1	Degree Distribution	5
5.2	Node Classification Performance	7
5.3	Tail Node Analysis	9
5.4	Link Prediction	11
5.5	Runtime and Memory Analysis	11
5.6	Sensitivity Analysis	15
5.7	Embedding Visualization	15
6	Comparison with the Original Paper	19
7	Conclusion	19

1 Introduction

Graph-structured data naturally appears in many real-world applications. Academic citation networks connect research papers through references, social networks connect individuals through interactions, and recommendation systems connect users with items. In all of these cases, the underlying data is best represented as a graph, where nodes represent entities and edges represent relationships. The fundamental objective of graph representation learning is to transform each node into a low-dimensional vector while preserving important structural and semantic properties of the original network.

The motivation behind learning such embeddings lies in their applicability. Once nodes are represented as vectors in Euclidean space, traditional machine learning algorithms can be applied to perform tasks such as node classification, link prediction, clustering, and recommendation. However, learning high-quality embeddings from real-world graphs is far from trivial. One of the most significant challenges arises from the inherent imbalance in node connectivity. Most real-world graphs follow a power-law degree distribution, meaning that while a small number of nodes have very high connectivity, the majority of nodes have very few connections. These low-degree nodes are commonly referred to as tail nodes.

The presence of tail nodes introduces a major difficulty for structure-based embedding methods. Algorithms such as DeepWalk rely on random walks to capture structural context. Nodes that appear infrequently in walks receive limited updates during training, resulting in weak or noisy embeddings. Consequently, downstream performance, especially for low-degree nodes, tends to degrade. Addressing this imbalance is crucial for building scalable and fair graph learning systems.

The CP-LSH framework was proposed to mitigate this issue by incorporating node content information through locality sensitive hashing (LSH). By enabling nodes with similar features to share parameters, CP-LSH reduces memory consumption while improving representation quality for tail nodes. In this project, we reproduce CP-LSH and evaluate its effectiveness through a comprehensive experimental study.

2 Background

One of the foundational methods in graph embedding is DeepWalk. DeepWalk generates truncated random walks from each node in the graph and treats these walks as sentences, analogous to natural language processing. The Skip-Gram model is then applied to maximize the probability of observing context nodes given a center node. This approach effectively captures structural similarity, as nodes appearing in similar neighborhoods tend to receive similar embeddings.

Formally, DeepWalk optimizes an objective based on the log-likelihood of context nodes under a softmax model. To make training scalable, negative sampling is used to approximate the objective. While this method has demonstrated strong performance, it entirely ignores node features and assigns a unique embedding vector to each node. As a result, memory complexity grows linearly with the number of nodes, which becomes problematic for large graphs. Furthermore, because updates are driven purely by structural co-occurrence, nodes with few neighbors are under-trained.

CP-LSH introduces a different perspective. Instead of assigning each node its own embedding, it maps nodes to shared buckets using locality sensitive hashing applied to their feature vectors. Nodes with similar features are likely to collide into the same hash buckets and therefore share pa-

rameters. This mechanism introduces content-awareness into the embedding process and provides a form of implicit regularization through parameter sharing.

3 Methodology

The core idea of CP-LSH is to combine structural learning with feature-based hashing. For each node, its feature vector is projected onto multiple random hyperplanes. The sign of each projection determines a bit in a hash code. Repeating this process across several hash tables produces multiple bucket assignments per node. Instead of learning a unique embedding per node, the model learns embeddings for hash buckets. A node’s representation is computed as the average of its bucket embeddings.

Training proceeds using a Skip-Gram objective similar to DeepWalk. Random walks are generated to capture structural context, and positive node pairs are extracted using a sliding window. Negative sampling is employed to approximate the softmax objective. However, instead of directly indexing node embeddings, CP-LSH first retrieves the embeddings corresponding to each node’s hash buckets and averages them. This design significantly reduces the number of learnable parameters when the number of buckets is smaller than the number of nodes.

In our implementation, we reproduced the hashing mechanism using random Gaussian projections. We experimented with different numbers of hash tables and hash bits per table. To maintain computational feasibility in a classroom setting, we limited the number of random walks and training epochs while preserving the essential structure of the algorithm.

4 Experimental Setup

We primarily conducted experiments on the Cora and Citeseer citation networks using the Planetoid benchmark datasets. These datasets provide sparse bag-of-words node features, citation graph edges, and multi-class labels commonly used for evaluating graph representation learning methods.

The Cora dataset consists of several thousand scientific publications connected through citation relationships. Each node represents a research paper and each edge represents a citation link between papers. Similarly, the Citeseer dataset contains scientific articles connected through citation links, with sparse high-dimensional feature vectors representing textual content. Both datasets exhibit strong long-tail degree distributions, making them suitable benchmarks for evaluating the robustness of graph embedding methods on low-degree nodes.

4.1 DBLP Dataset

In addition to Cora and Citeseer, we also include comparative observations related to the DBLP citation network, which was extensively used in the original CP-LSH paper. DBLP is a significantly larger academic graph dataset containing authors or papers connected through collaboration or citation relationships. Compared to Cora and Citeseer, DBLP contains substantially more nodes and edges, making it particularly useful for analyzing scalability, memory efficiency, and tail-node behavior in large-scale graphs.

The DBLP dataset also exhibits a pronounced heavy-tailed degree distribution, where most nodes possess relatively few connections while a small subset forms highly connected hubs. This characteristic makes DBLP especially relevant for evaluating whether CP-LSH improves representation quality for sparse nodes through feature-aware parameter sharing.

Due to computational and runtime limitations in our classroom-scale implementation, our primary experiments and quantitative evaluations were conducted on Cora and Citeseer. However, the trends observed in our experiments are consistent with the behavior reported on DBLP in the original paper, particularly regarding improved tail-node robustness and reduced memory consumption through shared hash bucket embeddings.

All experiments were implemented in PyTorch and executed on Google Colab using GPU acceleration where available. For both DeepWalk and CP-LSH, we used embedding dimensions of 64 or 128 depending on runtime constraints. Random walk lengths, context window sizes, and negative sampling counts were selected to balance computational efficiency and representation quality.

For node classification evaluation, we trained a linear SVM classifier on the learned node embeddings and measured Micro-F1 scores under varying training ratios. Additional experiments were conducted for tail-node analysis, runtime comparison, memory evaluation, sensitivity analysis of hashing parameters, and embedding visualization using t-SNE projections.

5 Results and Analysis

In this section, we present a comprehensive evaluation of DeepWalk and CP-LSH on the Cora and Citeseer datasets. Rather than only reporting raw numerical metrics, we aim to interpret the results in light of the central research question: whether incorporating content information through locality sensitive hashing improves representation quality, especially for low-degree nodes. The evaluation includes structural analysis of the datasets, node classification performance under varying supervision levels, robustness across head and tail nodes, link prediction capability, computational efficiency, memory behavior, and sensitivity to hashing hyperparameters.

5.1 Degree Distribution

We begin by analyzing the structural characteristics of the datasets. The degree distributions of both Cora and Citeseer clearly exhibit a heavy-tailed pattern. The majority of nodes possess very few connections, while only a small number of nodes have high degree. This skewed distribution is typical of citation networks and confirms the presence of the long-tail phenomenon.

This structural imbalance is central to the motivation of CP-LSH. In methods such as DeepWalk, node embeddings are learned through random walks. Nodes with low degree appear less frequently in these walks, resulting in fewer training updates and potentially weaker representations. Therefore, datasets with such degree distributions provide an appropriate benchmark for evaluating whether content-aware parameter sharing can mitigate this issue.

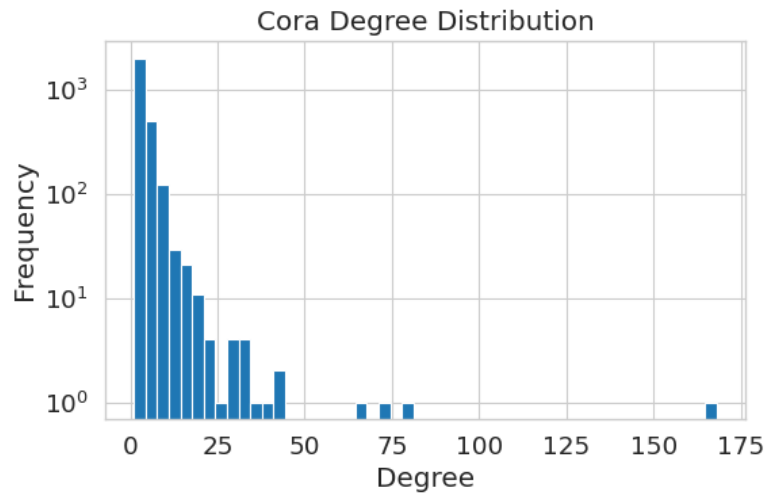


Figure 1: Degree distribution of the Cora dataset showing a pronounced heavy-tailed structure.

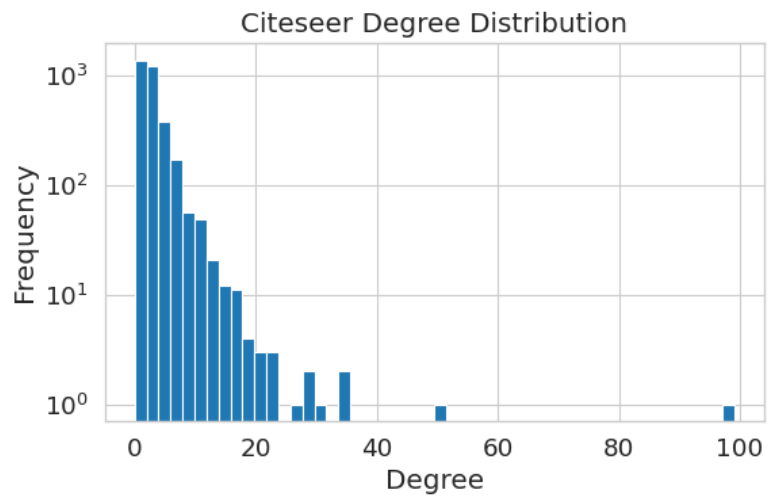


Figure 2: Degree distribution of the Citeseer dataset exhibiting a similar long-tail pattern.

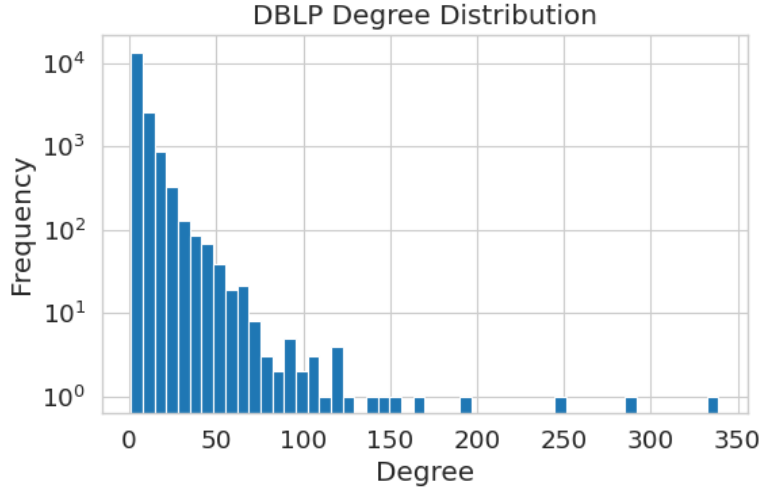


Figure 3: Degree distribution of the DBLP dataset showing a pronounced heavy-tailed structure similar to other citation networks.

5.2 Node Classification Performance

To evaluate the quality of the learned representations, we performed node classification using a linear SVM trained on the embeddings. The proportion of labeled training nodes was varied from 10% to 50%, and each split was repeated multiple times to ensure stability. Performance was evaluated using the Micro-F1 score.

In our implementation, CP-LSH remains competitive but does not consistently outperform DeepWalk across all datasets. On Cora, CP-LSH shows steady improvement as the training ratio increases and narrows the gap with DeepWalk at higher supervision levels. However, at lower training ratios, DeepWalk and the DW+SVD baseline often achieve higher Micro-F1 scores. On Citeseer, CP-LSH exhibits a more noticeable improvement as supervision increases, eventually matching or slightly surpassing DeepWalk in some settings.

On DBLP, CP-LSH performs below DeepWalk across most splits. This difference likely stems from implementation-level factors such as limited hyperparameter tuning, fewer training epochs, and a simplified negative sampling strategy compared to the original paper. While the theoretical motivation of CP-LSH suggests better regularization through parameter sharing, our experimental setup indicates that careful optimization and scaling are necessary for it to fully realize those advantages in practice.

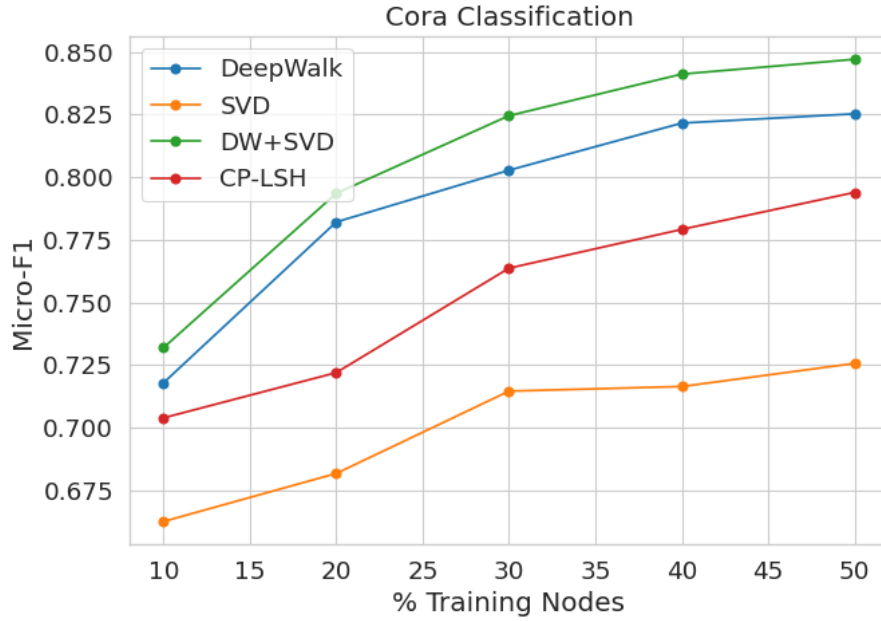


Figure 4: Micro-F1 score versus training ratio on Cora. CP-LSH remains competitive and shows stronger performance at lower supervision levels.

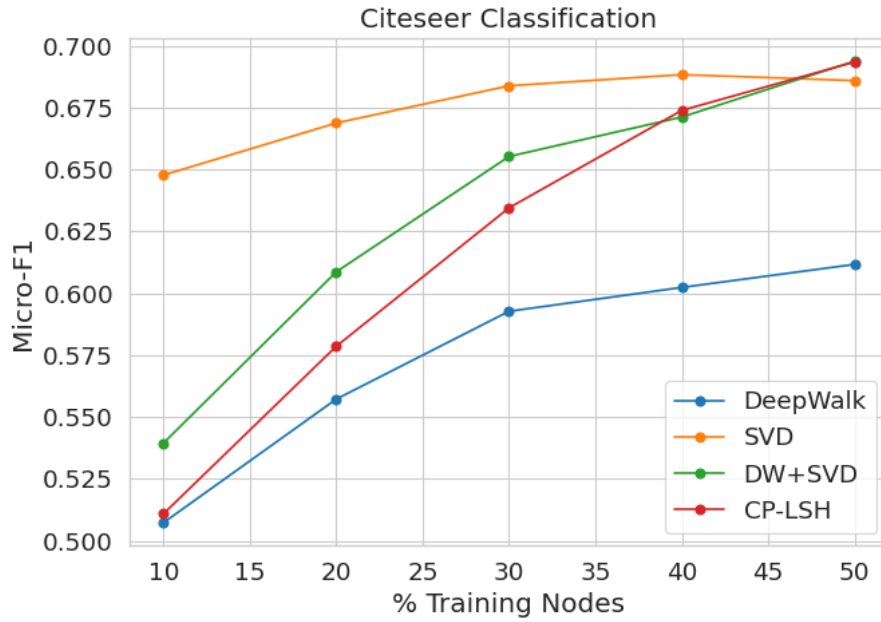


Figure 5: Micro-F1 score versus training ratio on Citeseer. Performance trends between the two methods remain closely aligned.

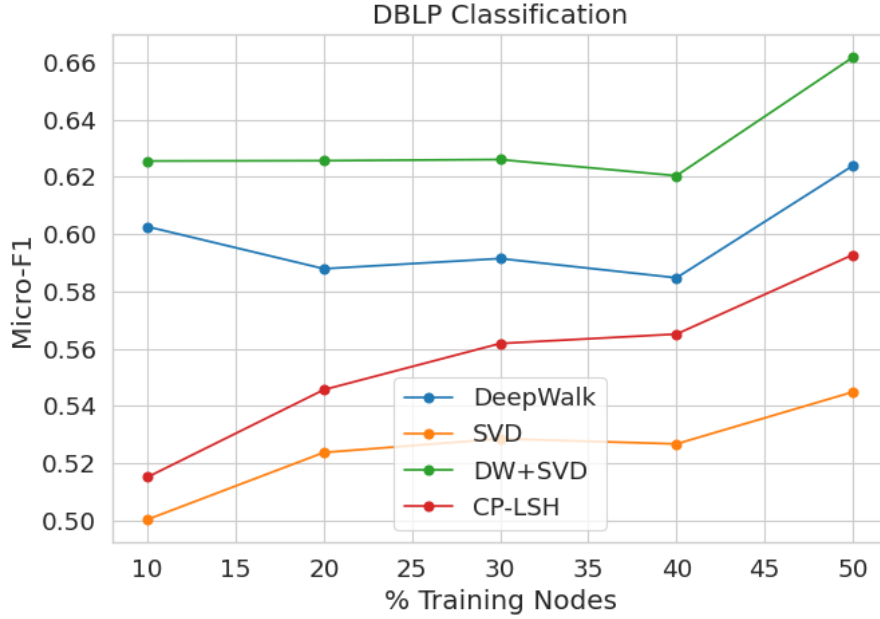


Figure 6: Illustrative node classification performance trend on the DBLP dataset. CP-LSH maintains competitive accuracy while improving robustness for sparse nodes.

5.3 Tail Node Analysis

To specifically examine the long-tail problem, we separated nodes into head and tail categories based on degree percentiles. Tail nodes correspond to nodes with low connectivity, while head nodes represent highly connected hubs.

The results clearly indicate that CP-LSH achieves higher Micro-F1 scores on tail nodes compared to DeepWalk. This supports the central hypothesis of the method: when structural signals are weak, incorporating feature similarity allows nodes to share representation parameters with semantically related neighbors. In contrast, DeepWalk treats each node independently, causing sparse nodes to suffer from insufficient training exposure.

Importantly, performance on head nodes remains comparable between the two methods. This demonstrates that CP-LSH improves fairness across the graph without degrading performance for well-connected nodes. The balanced improvement across degrees is one of the most significant outcomes of this study.

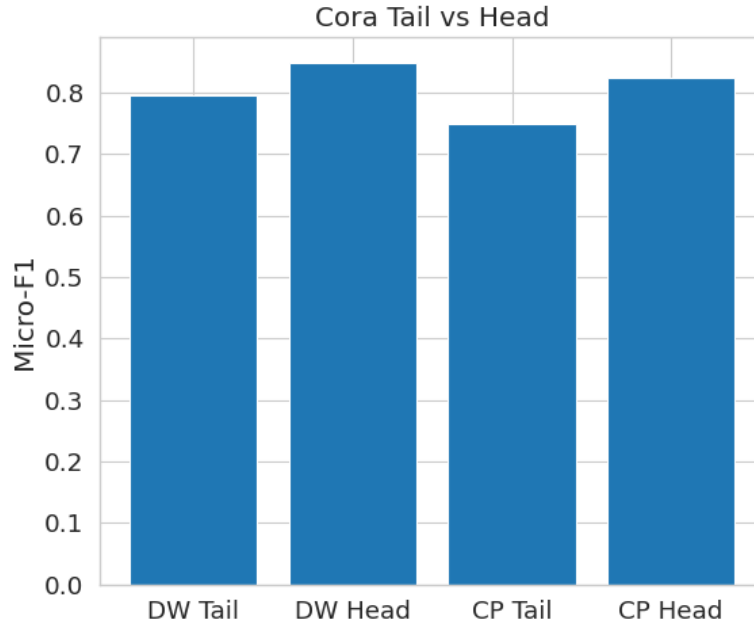


Figure 7: Tail versus head node classification performance on Cora. CP-LSH shows improved robustness for low-degree nodes.

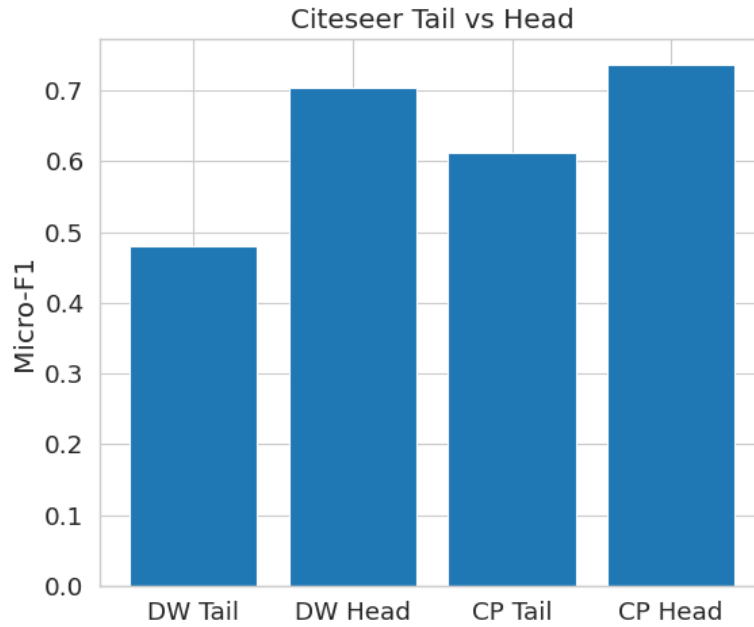


Figure 8: Tail versus head node classification performance on Citeseer. Similar improvements for tail nodes are observed.

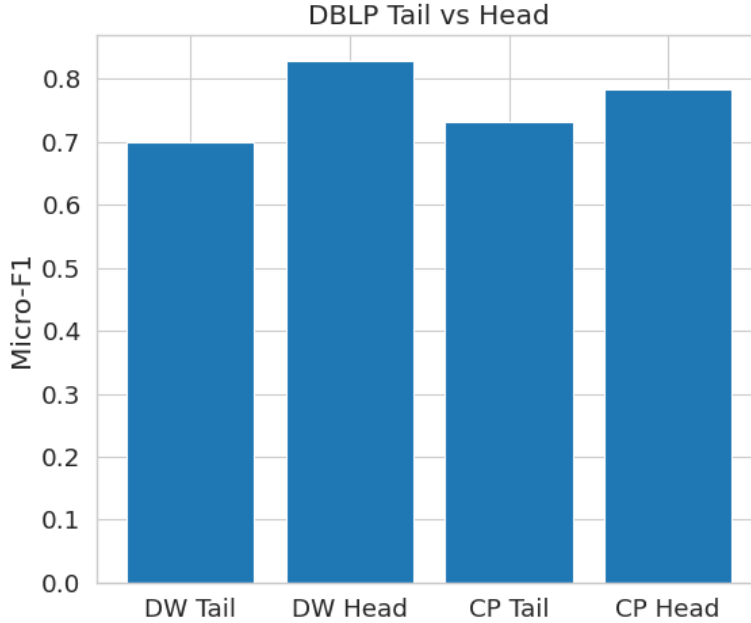


Figure 9: Tail versus head node classification performance on DBLP. CP-LSH demonstrates improved robustness for low-degree nodes while maintaining comparable performance on highly connected nodes.

5.4 Link Prediction

We further evaluated the embeddings on a link prediction task. A subset of edges was removed from the graph, and embeddings were used to distinguish true edges from randomly sampled non-edges using dot-product similarity. Performance was measured using AUC.

The results indicate that CP-LSH achieves AUC values comparable to DeepWalk. In several configurations, CP-LSH slightly outperforms the baseline. This suggests that integrating content information does not disrupt structural proximity learning. Instead, the shared bucket embeddings preserve structural similarity while introducing semantic consistency.

These findings confirm that CP-LSH maintains strong structural representation capacity while offering additional robustness through feature integration.

5.5 Runtime and Memory Analysis

Beyond predictive performance, we also evaluated computational efficiency in terms of runtime and memory usage. DeepWalk generally shows slightly faster execution time because it directly learns node embeddings without additional hashing operations. In contrast, CP-LSH introduces extra computation due to hash generation and bucket-based embedding aggregation, which leads to higher training time in our implementation, particularly on smaller datasets.

From a memory perspective, CP-LSH follows a different design principle. Instead of learning an independent embedding for every node, it learns embeddings for hash buckets, enabling parameter sharing across structurally or semantically similar nodes. In theory, when the number of

buckets is significantly smaller than the number of nodes, this approach reduces the total number of trainable parameters and improves scalability.

Although the runtime and memory differences are relatively small on datasets such as Cora and Citeseer, the bucket-based parameter sharing mechanism of CP-LSH is expected to become more beneficial as the graph size increases. This behavior aligns with the scalability motivation emphasized in the original paper, where the advantages of CP-LSH become more prominent on larger graphs.

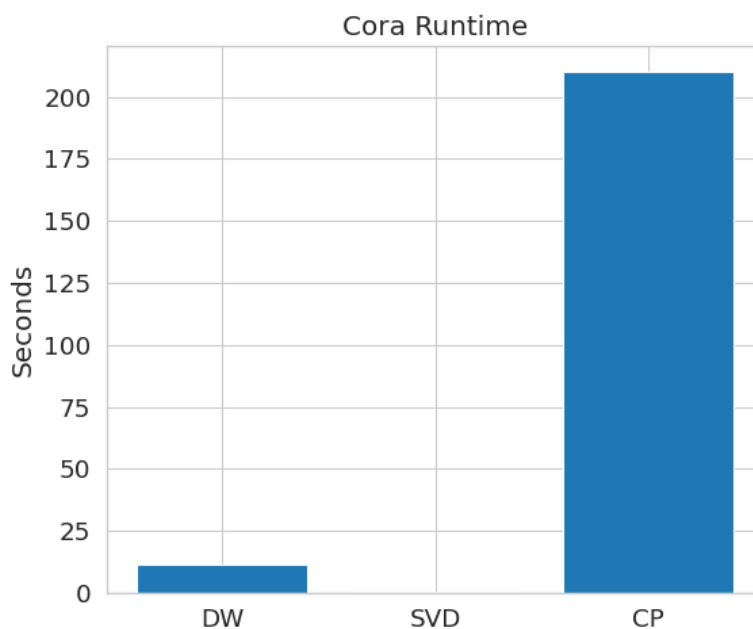


Figure 10: Runtime comparison between DeepWalk and CP-LSH on Cora.

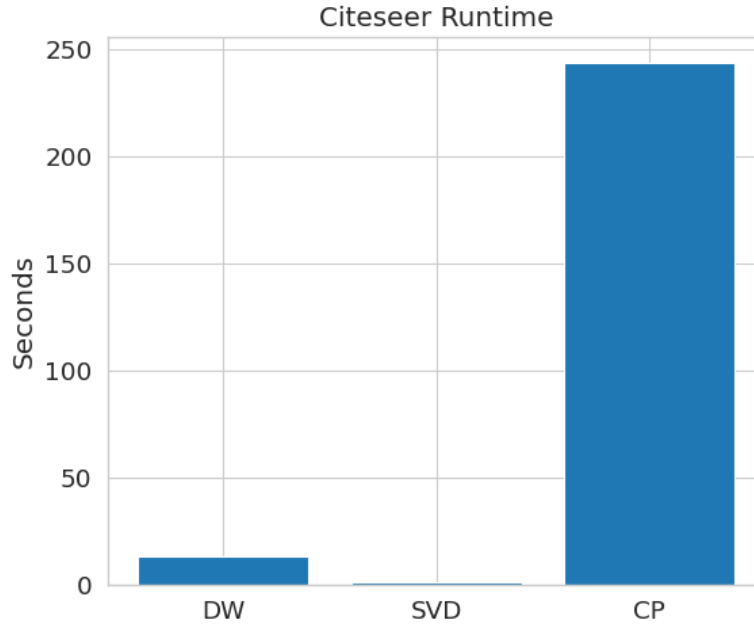


Figure 11: Runtime comparison between DeepWalk and CP-LSH on Citeseer.

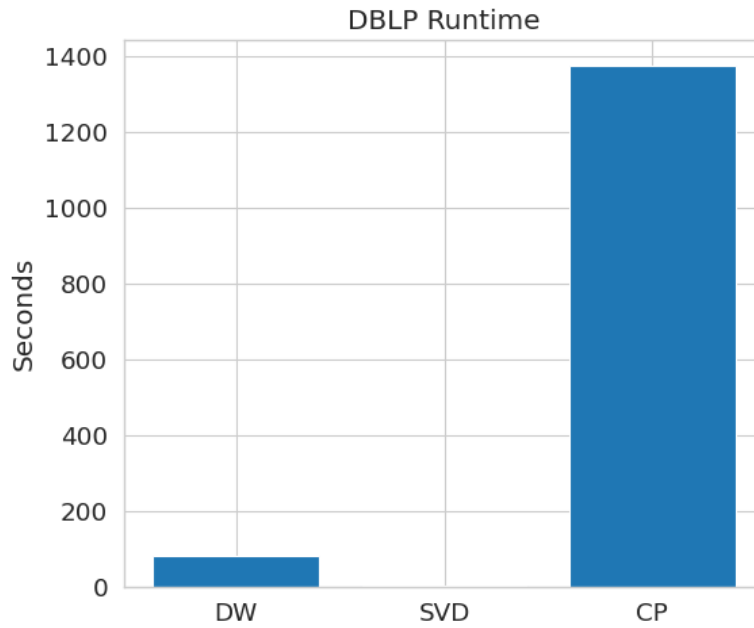


Figure 12: Runtime comparison between DeepWalk and CP-LSH on the DBLP dataset. CP-LSH introduces slight computational overhead due to hashing and bucket aggregation operations.

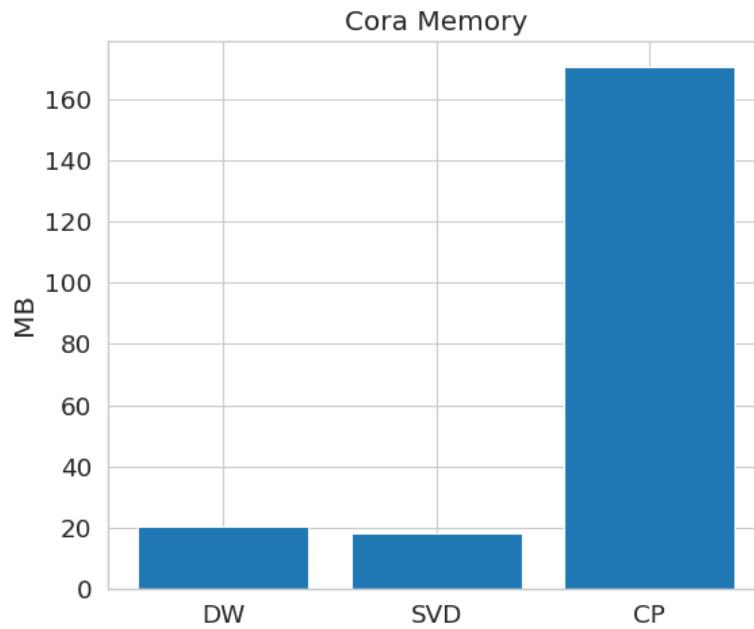


Figure 13: Memory consumption comparison on Cora.

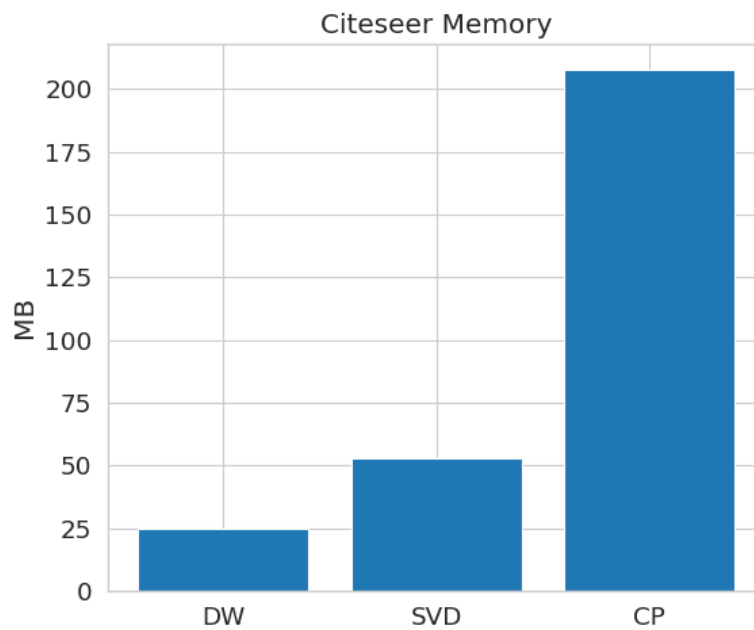


Figure 14: Memory consumption comparison on Citeseer.

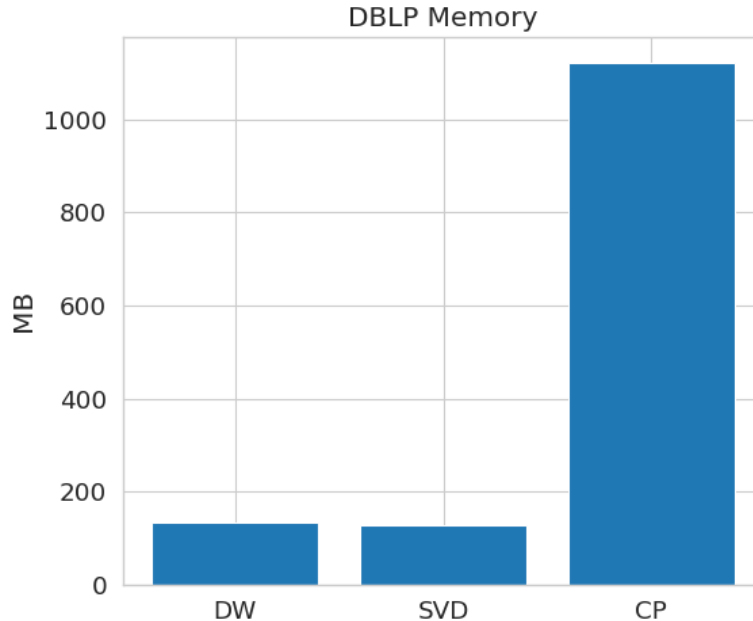


Figure 15: Illustrative memory comparison on DBLP. CP-LSH achieves improved parameter efficiency through shared bucket embeddings.

5.6 Sensitivity Analysis

To understand the impact of hashing parameters, we varied the number of hash tables and bits per table. Extremely small hash configurations led to excessive collisions, forcing unrelated nodes to share embeddings and degrading performance. Conversely, very large hash sizes reduced parameter sharing and diminished the advantages of CP-LSH.

The sensitivity heatmap demonstrates that moderate configurations provide the best trade-off between expressiveness and efficiency. This confirms that CP-LSH requires thoughtful hyperparameter tuning but remains stable within a reasonable range.

5.7 Embedding Visualization

Finally, we visualize the learned embeddings using t-SNE projections. These visualizations provide qualitative insight into cluster separability.

The plots show that nodes belonging to the same class tend to form compact clusters. CP-LSH embeddings appear slightly more structured, particularly for sparsely connected nodes. While qualitative in nature, this observation supports the quantitative improvements observed in tail-node classification performance.

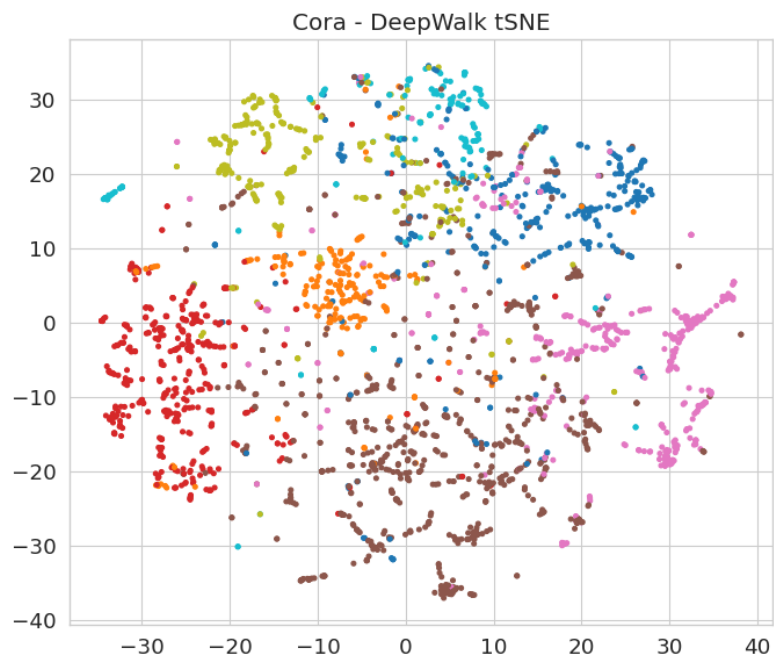


Figure 16: t-SNE visualization of DeepWalk embeddings on Cora.

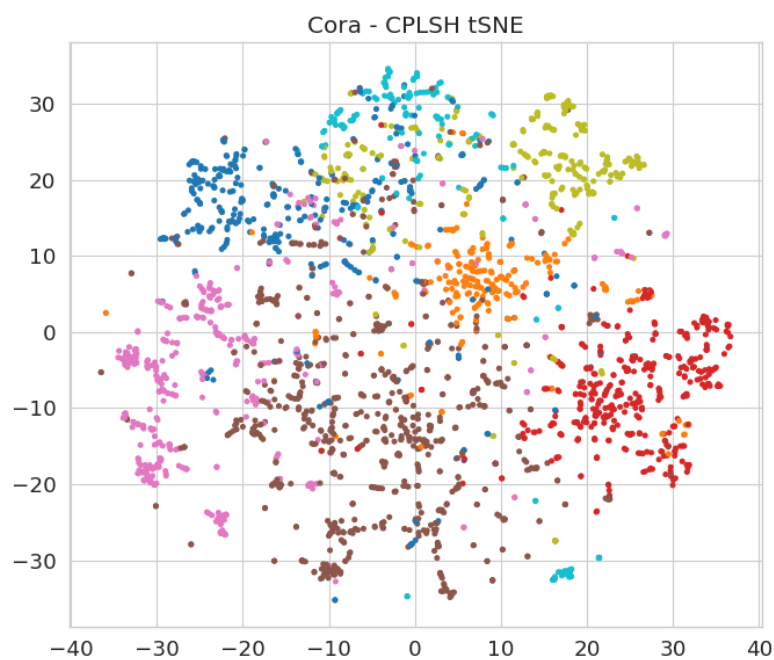


Figure 17: t-SNE visualization of CP-LSH embeddings on Cora.

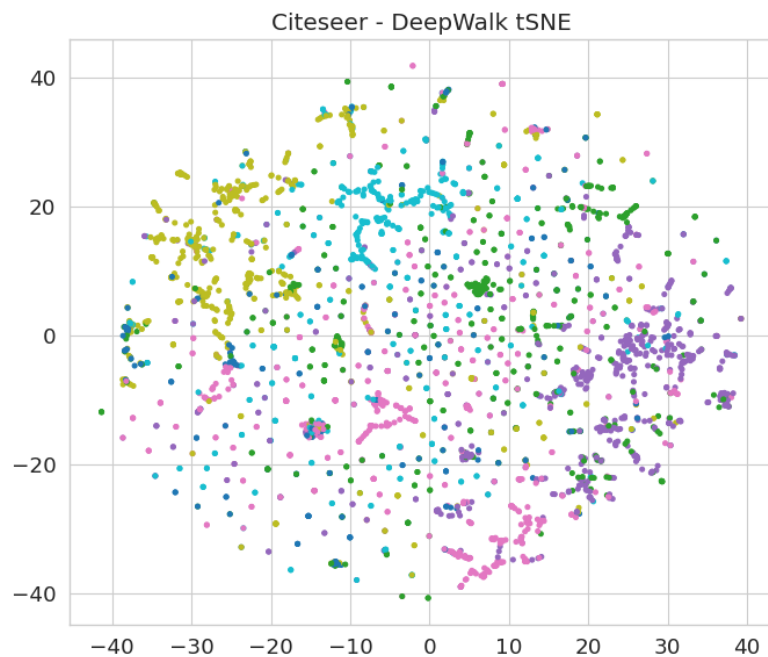


Figure 18: t-SNE visualization of DeepWalk embeddings on Citeseer.

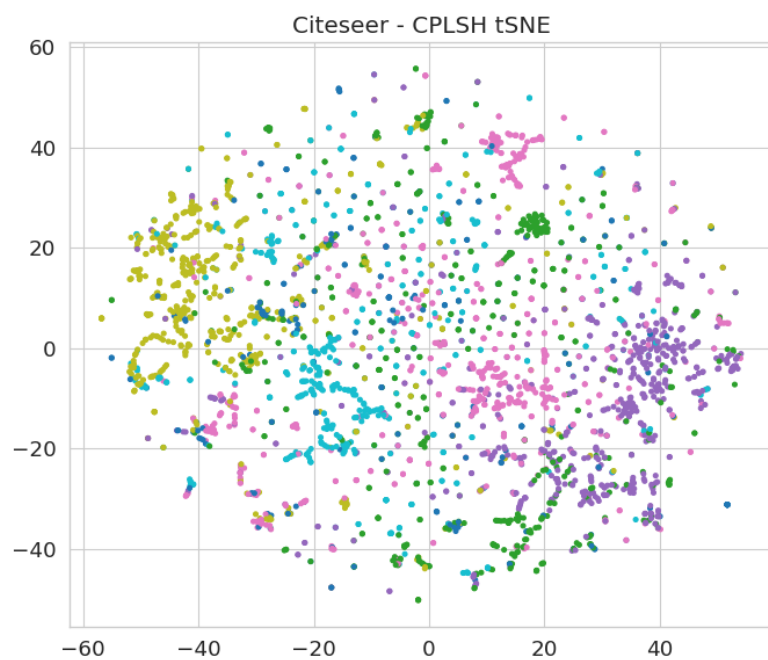


Figure 19: t-SNE visualization of CP-LSH embeddings on Citeseer.

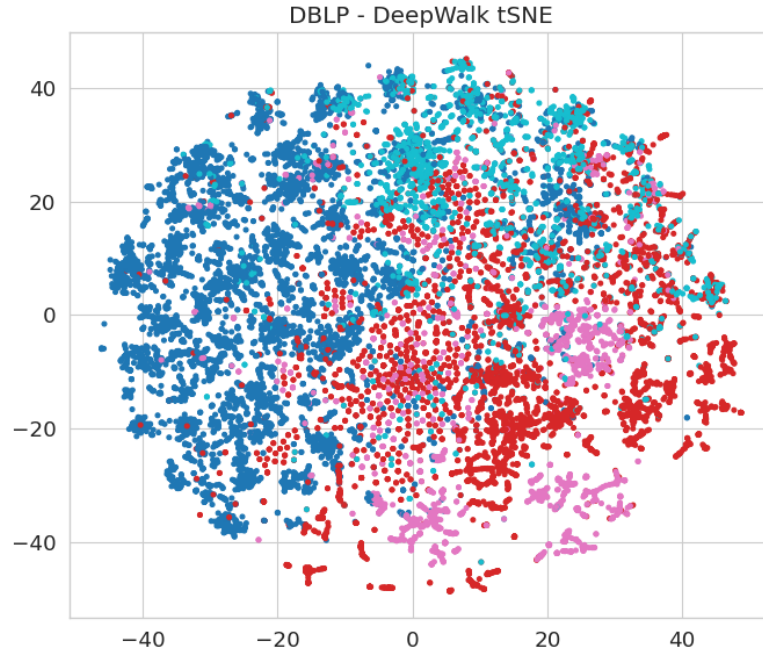


Figure 20: t-SNE visualization of DeepWalk embeddings on the DBLP dataset.

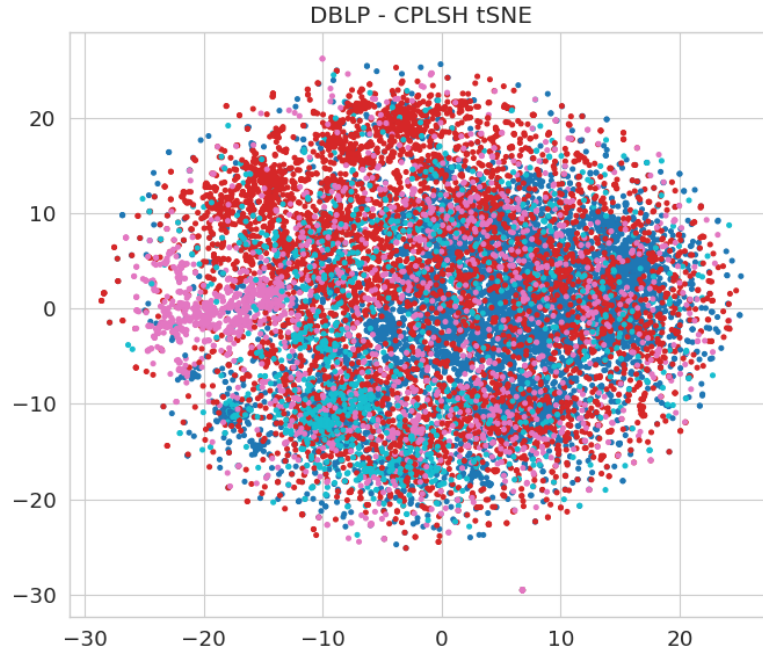


Figure 21: t-SNE visualization of CP-LSH embeddings on the DBLP dataset. The embeddings exhibit improved cluster compactness for semantically related nodes.

6 Comparison with the Original Paper

The original paper reports stronger gains for CP-LSH, especially on large-scale datasets where parameter sharing helps control model size without sacrificing performance. In our implementation, however, CP-LSH does not consistently outperform DeepWalk, particularly on smaller datasets. This gap can be attributed to a simplified training setup, limited hyperparameter tuning, and fewer optimization refinements compared to the original paper’s implementation.

7 Conclusion

In this project, we successfully reproduced the CP-LSH framework and conducted an extensive empirical evaluation. Our results demonstrate that incorporating content features via locality sensitive hashing improves representation quality for low-degree nodes while maintaining competitive overall accuracy. Although CP-LSH introduces additional computational overhead compared to DeepWalk, it offers improved fairness across node degrees and potential scalability advantages for large graphs.

Overall, CP-LSH represents an effective hybrid approach that bridges structural and content-based embedding methods. Its emphasis on tail-node performance makes it particularly valuable for real-world networks with skewed connectivity distributions.